

课程笔记

[LLM Agents F24] LLM Reasoning — Denny Zhou

基于公开课程资料整理

2024 年 9 月 9 日



DENNY ZHOU
PRINCIPAL SCIENTIST /
RESEARCH DIRECTOR



视频作者/频道: Berkeley RDI

发布日期: 2024 年 9 月 9 日

视频时长: 约 60 分钟

视频链接: https://www.youtube.com/watch?v=QL-FS_Zcmyo

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：为什么推理能力是 AI 的关键	2
1.1 Last Letter Concatenation：一个简单但深刻的例子	2
1.2 本章小结	2
2 中间步骤 (Intermediate Steps)：推理的核心机制	2
2.1 历史脉络：从训练到提示	2
2.2 理论基础：为什么中间步骤有效	3
2.3 本章小结	3
3 推理策略：从 Few-shot 到 Zero-shot	3
3.1 Least-to-Most Prompting：分解复杂任务	3
3.2 Zero-shot CoT：“Let’s think step by step”	4
3.3 LLMs as Analogical Reasoners	4
3.4 Chain-of-Thought Reasoning without Prompting	4
3.5 本章小结	4
4 Self-Consistency：从采样到投票	4
4.1 动机：LLM 的概率本质	4
4.2 方法与效果	5
4.3 本章小结	5
5 推理的局限性	5
5.1 LLM 容易被无关上下文干扰	5
5.2 LLM 无法自我纠错推理	6
5.3 Multi-Agent Debate 并不优于 Self-Consistency	6
5.4 前提顺序 (Premise Order) 影响推理	6
5.5 本章小结	6
6 总结与延伸	6
6.1 核心要点回顾	6
6.2 对 LLM Agent 的启示	7
6.3 拓展阅读	7

1 引言：为什么推理能力是 AI 的关键

Denny Zhou 是 Google DeepMind 的研究科学家，长期专注于大语言模型的推理能力研究。他是 Chain-of-Thought Prompting 和 Self-Consistency 等里程碑工作的核心贡献者。本次讲座从一个根本性问题出发——**什么才算真正的智能？**——引出 LLM 推理的核心脉络。

从机器学习到推理的范式转变

传统机器学习 (few-shot learning、active learning、meta-learning 等) 追求数据高效性 (data efficiency)，但在实践中这些方法并没有真正实现“从少量样本学习”。Denny Zhou 认为，人类能够从少量样本中学习，根本原因不是统计规律，而是**推理能力 (reasoning)**。这一洞察驱动了他从机器学习转向 LLM 推理的研究方向。

1.1 Last Letter Concatenation：一个简单但深刻的例子

Denny Zhou 用 Last Letter Concatenation 任务来揭示传统方法与推理方法的巨大差异：

- **任务定义**：给定一个人名（如 “Elon Musk”），输出名和姓最后一个字母的拼接（“n” + “k” = “nk”）。
- **传统 ML 方法**：用 Transformer 训练，需要数千标注样本，准确率约 85–90%。
- **Few-shot Prompting**：直接给 LLM 几个示例，LLM 仍然容易出错（如 “Barack Obama” 输出 “ck” 而非 “ka”）。
- **Chain-of-Thought Prompting**：在示例中加入推理步骤（“The last letter of Elon is n, the last letter of Musk is k, so nk”），仅用一个示例即可达到 100% 准确率。

核心洞察

对于人类来说简单的任务，如果一个方法需要海量标注数据才能学会，那它难以称为真正的「智能」。推理能力使 LLM 能够从极少样本中泛化，这是传统机器学习方法无法比拟的。

1.2 本章小结

推理能力是弥合「数据驱动学习」与「真正智能」之间鸿沟的关键。LLM 通过 Chain-of-Thought 式的中间步骤生成，实现了质的飞跃——从需要大量训练数据降低到仅需少量甚至零样本。

2 中间步骤 (Intermediate Steps)：推理的核心机制

2.1 历史脉络：从训练到提示

Denny Zhou 梳理了中间步骤思想的发展历程：

1. **2017 年, Ling et al.**：在论文中提出用自然语言推理步骤 (rationale) 求解数学题，训练 seq2seq 模型从头生成中间步骤再得出答案。被 Denny Zhou 称为“如同时间旅行者”。
2. **2021 年, OpenAI GSM8K**：延续 2017 年思路，创建了包含中间步骤的数学数据集，用于 fine-tune GPT-3，大幅提升数学推理能力。

3. **2021 年, Google Brain Scratchpad**: 独立发现类似思路, 但在程序合成 (program synthesis) 领域使用符号化中间步骤。
4. **2022 年, Chain-of-Thought Prompting (Wei et al.)**: 在 prompting 阶段广泛评估中间步骤的效果, 展示了在几乎所有 NLP 任务上的显著提升。

中间步骤是关键, 而非具体方法

无论是 training、fine-tuning 还是 prompting, 真正起作用的是**中间步骤 (intermediate steps)** 本身。当提供包含中间步骤的示例时, LLM 会生成同样包含中间步骤的响应——这是 LLM 作为概率模型模仿输入模式的自然结果。

2.2 理论基础: 为什么中间步骤有效

Denny Zhou 引用了 2024 年与 Stanford 的 Branden Srouji 合作的理论工作:

- **带中间步骤的 Transformer**: 只要深度超过一个**常数** (与输入长度无关), 就可以解决任何固有串行 (inherently serial) 问题。
- **直接输出答案的 Transformer**: 要么需要巨大的深度, 要么根本无法求解。

实践含义

如果模型无法解决某个问题, 一个有效策略是引导它生成更多的中间步骤。此外, 可以调用外部工具来辅助中间步骤的生成——这正是 LLM Agent 框架的核心思想之一。

2.3 本章小结

中间步骤思想经历了从训练到 fine-tuning 再到 prompting 的演进, 但本质始终不变: 让模型逐步推导而非直接跳到答案。理论上, 中间步骤赋予了有限深度 Transformer 解决任意串行计算的能力。

3 推理策略: 从 Few-shot 到 Zero-shot

3.1 Least-to-Most Prompting: 分解复杂任务

受 Pólya 经典著作《How to Solve It》中分解策略的启发, Denny Zhou 团队提出了 Least-to-Most Prompting:

- **核心思想**: 先将复杂问题分解为一系列子问题, 然后从最简单的子问题开始逐步求解, 每个子问题的解作为后续子问题的上下文。
- **SCAN 任务**: 组合泛化 (compositional generalization) 基准, 仅用 1% 的示例即达到 99.7% 准确率。
- **Text-to-Code 任务**: 使用 Dynamic Least-to-Most Prompting, 仅用 1% 的数据即大幅超越使用全部训练数据的专用架构。

组合泛化 (Compositional Generalization)

指测试样本比训练/提示样本更复杂的场景。例如，训练时只见过短代码片段，测试时需要生成更长的代码。Least-to-Most Prompting 通过分解策略天然地处理了这一问题。

3.2 Zero-shot CoT: “Let’s think step by step”

- 无需任何示例，仅在问题后加一句 “Let’s think step by step” 即可触发 LLM 的逐步推理。
- 效果通常不如 few-shot CoT，但胜在零成本。

3.3 LLMs as Analogical Reasoners

受 Pólya 的类比推理思想启发：

- **方法**：面对新问题时，让 LLM 自行生成相关问题及其解法，再利用这些自生成的示例来解决原问题。
- **优势**：比 “Let’s think step by step” 显著更好，甚至超越手工设计的 few-shot 示例——因为模型会为每个问题生成定制化的相关示例。
- **与检索增强的关系**：可以进一步扩展为从网络检索相关问题和知识，实现 scaling。

3.4 Chain-of-Thought Reasoning without Prompting

一个令人惊讶的发现：**无需任何提示**，仅通过特殊解码策略即可触发推理：

- 在解码第一步，不取概率最高的 token，而是探索 top- k 个候选 token。
- 对每个候选 token 继续贪心解码，生成完整回答。
- 选择包含推理步骤且最终答案置信度最高的路径。

关键观察

LLM 内部已经“知道”如何推理。当推理路径（如 “Nicolas Cage was born in 1964, and 1964 is an even year”）出现时，最终答案的概率从低值跃升至 98%。推理步骤在概率层面显著提升了模型对最终答案的置信度。

3.5 本章小结

从 few-shot CoT 到 zero-shot CoT，再到无需提示的特殊解码策略，LLM 推理能力的触发方式越来越灵活。核心规律是：推理步骤本身而非触发方式才是关键。类比推理进一步表明，LLM 可以自主生成相关示例来辅助推理。

4 Self-Consistency: 从采样到投票

4.1 动机：LLM 的概率本质

Denny Zhou 从第一性原理出发推导 Self-Consistency 的合理性：

- LLM 在解码时优化的是： $\arg \max P(\text{reasoning path}, \text{answer} \mid \text{problem})$
- 但我们真正需要的是： $\arg \max P(\text{answer} \mid \text{problem})$
- 两者的关系： $P(\text{answer} \mid \text{problem}) = \sum_{\text{path}} P(\text{answer}, \text{path} \mid \text{problem})$
- 即需要对所有可能的推理路径求和（marginalize），而非只取单一最优路径。

4.2 方法与效果

Self-Consistency 方法

1. 对同一问题采样多次（使用非零温度），获得多条推理路径和对应答案。
2. 取出现频率最高的答案作为最终预测（多数投票 / majority voting）。
3. 注意：投票对象是**最终答案**，而非推理路径——推理路径是隐变量（latent variable）。

Self-Consistency 带来了巨大的性能提升，在当时大幅刷新了所有基准的 SOTA。更重要的是，当一致性超过 80% 时，准确率接近 100%——这提供了一个可靠的置信度指标。

与 Universal Self-Consistency 的扩展

对于自由形式（free-form）答案，可以通过 LLM 自身来判断哪些答案语义等价，找出最常见的响应聚类。例如对于“哪些国家人均咖啡消费低于墨西哥”这类问题，不同回答可能措辞不同但内容一致。

4.3 本章小结

Self-Consistency 将 LLM 推理从单次生成提升到统计推断层面。其核心思想源自概率图模型中的边际化推理（marginal inference），简单但极为有效。

5 推理的局限性

5.1 LLM 容易被无关上下文干扰

- 在数学题中插入无关信息（如“Mario’s moustache costs \$10”），LLM 会被误导。
- 添加提示“ignore irrelevant context”可以部分缓解，但当无关内容容量增大时效果有限。
- 即使是简单的无关句子（“the sky is blue”）大量堆积，也会导致显著的性能下降。

上下文干扰

LLM 作为概率模型，会将所有输入 token 纳入注意力计算。无关信息不仅浪费上下文窗口，还会实质性地干扰推理过程。这在实际的 Agent 应用中需要特别注意——检索增强（RAG）引入的噪声文档可能反而损害推理质量。

5.2 LLM 无法自我纠错推理

Denny Zhou 团队的实验表明：

- 让 LLM 审查并修正自己的答案时，它可能纠正错误答案，但也会把正确答案改错。
- 在 GSM8K、CommonsenseQA 等基准上，self-correction 方法没有带来任何净提升，反而使性能变差。
- 文献中报告的 self-correction 提升往往使用了 Oracle (即只在答案错误时才要求模型修正)——但模型本身无法判断自己是否正确。

外部反馈是自我纠错的前提

LLM 的自我纠错需要外部 Oracle 反馈 (如代码任务中的 unit test)。Self-Debug 工作通过单元测试自然地提供了这种 Oracle。纯粹依赖模型自身判断来纠错是不可靠的。

5.3 Multi-Agent Debate 并不优于 Self-Consistency

- 多个 Agent 互相辩论并达成共识 (multi-agent debate)，总共生成 n 个回答。
- 但简单地对 n 个独立采样使用 Self-Consistency 投票，效果始终优于 multi-agent debate。
- 辩论过程引入的信息交互并未带来额外收益。

5.4 前提顺序 (Premise Order) 影响推理

- **GSM8K 实验**：仅将问题中的句子重新排列 (不改变语义)，准确率下降约 10 个百分点。
- **逻辑推理实验**：使用随机符号的纯逻辑规则推理，仅打乱相关规则的顺序，所有 Frontier 模型准确率下降 30+ 个百分点。
- **根本原因**：LLM 只能顺序处理输入，无法像人类一样在前提之间自由跳转和回溯。

前提顺序效应

LLM 的推理严重依赖前提的呈现顺序。在设计 Agent 系统时，输入信息的组织顺序需要与推理所需的顺序对齐，否则会导致显著的性能损失。

5.5 本章小结

当前 LLM 推理存在三大局限：容易被无关上下文干扰、无法可靠自我纠错、对前提顺序敏感。这些局限性为 Agent 系统的设计提供了重要指导——需要通过外部工具和精心设计的工作流来弥补这些短板。

6 总结与延伸

6.1 核心要点回顾

1. **中间步骤**是提升 LLM 推理能力的核心机制，无论采用 training、fine-tuning 还是 prompting。

2. **Self-Consistency** 通过多次采样和多数投票，从概率角度显著提升推理准确率。
3. **推理策略** 从 few-shot CoT 到 zero-shot CoT，再到 analogical reasoning 和无提示解码，触发方式日趋灵活。
4. **局限性**——上下文干扰、自我纠错失败、前提顺序敏感——指明了未来改进方向。

6.2 对 LLM Agent 的启示

- **工具调用与外部反馈**：Agent 可以通过调用代码执行器、搜索引擎等工具提供外部 Oracle，弥补 LLM 无法自我纠错的短板。
- **检索增强需谨慎**：RAG 引入的文档可能包含无关信息，反而干扰推理；需要精准的检索和过滤。
- **信息组织**：Agent 工作流程中信息的呈现顺序应与推理逻辑对齐。
- **Scaling 推理**：Self-Consistency 和 Analogical Reasoning 可以作为 test-time compute scaling 的手段。

6.3 拓展阅读

- Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” NeurIPS 2022.
- Wang et al., “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” ICLR 2023.
- Zhou et al., “Least-to-Most Prompting Enables Complex Reasoning in Large Language Models,” ICLR 2023.
- Kojima et al., “Large Language Models are Zero-Shot Reasoners,” NeurIPS 2022.
- Yasunaga et al., “Large Language Models as Analogical Reasoners,” ICLR 2024.
- Huang et al., “Large Language Models Cannot Self-Correct Reasoning Yet,” ICLR 2024.
- Pólya, “How to Solve It,” Princeton University Press, 1945.